

Topic 2.3

2.3: The Type `size_t`

Often, we need to specify the size of things, and we need to hold those sizes in variables within our C program. Examples include the size of dynamically-allocated blocks of memory, the size of disk files, and the length of strings.

Remember that the basic types *int*, *long int*, etc. have a guaranteed minimum range of values, but the actual range is *system-dependent*.

If we arbitrarily choose the type *unsigned int* for the variables which hold the size of things, it may work on some systems, but not on others because of these range differences, as well as differences in available memory and other characteristics.

If we arbitrarily choose the type *unsigned long* for the variables which hold the size of things, such variables will almost always have enough bits to hold the size of things. However, they may have too many bits and be wasteful of space. Remember, *unsigned long* values must have a range from at least 0 to approximately 4.2 billion, which requires 32 bits. They may in fact have more than 32 bits (48, 64, etc.).

Because many standard C library functions have formal parameters of type *size_t*, it is vital that the corresponding actual parameters, often variables, are of the correct type.

To solve these problems, the type *size_t* is defined (using a *typedef*, which we'll get to soon) by the local implementation. In reality, any variable which is declared of type *size_t* is actually an *unsigned int* or an *unsigned long*, whichever is appropriate for the hardware, operating system, and compiler installation.

Whenever you need a variable that will hold "the size of something," declare it of type *size_t*. Your application will then be portable to other platforms without worrying about what range of values is actually provided on the other platforms for some particular type.

To illustrate this:

```
unsigned int  mem_needed; /* Not portable because it won't
                           always have enough bits */

unsigned long mem_needed; /* Not portable because it may have
                           too many bits */

/* Either of the above can cause type mismatches
   with standard library functions.*/

size_t       mem_needed; /* Portable! */
```

Topic 2.4 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-4>)